iHelp — Project Overview & Review Guide

Internet Technologies — Become a Full-Stack Engineer · RUNI CS 2026

This document stands in for a live presentation: it walks through everything a presentation would cover — what the product is, who it's for, how it's built, and every technical decision — and then gives a **5-minute hands-on review path** so you can confirm it works, with nothing to install.

- **Live site:** https://ihelp-roan.vercel.app
- **Code repository:** https://github.com/YuvGev00/ihelp

---

1. What the product is

iHelp is a reversed help-marketplace. Instead of a person who needs help searching for a provider and calling around, they **post a single request**, and identity-verified helpers nearby **compete** with offers — for pay or as volunteers. The requester picks one offer, both sides confirm the work is done, and the requester rates the helper. Demand is published once, and supply comes to it.

2. The problem it solves

Finding trustworthy, available help is slow and one-directional: you search, compare, and phone providers one by one, and they can't even see that you need help. Trust is hard to establish and has to run **both ways** — a fake request endangers a helper just as a fake helper endangers a requester. And small, volunteer-suitable tasks (a ride, help with a form) have willing neighbours but no channel connecting them. iHelp inverts the search and verifies both sides.

3. Who the users are

One account type — the same person can request help in the morning and offer help in the afternoon; permissions are attached to **rows, not accounts**. Three roles appear: a **requester**, a **verified helper** (optionally with a reviewed professional badge), and an **admin** who reviews verifications and moderates.

4. Why it has business value

The **customer** — the side that would be monetized — is the professional helper: iHelp is a lead-generation channel that delivers nearby, ready-to-buy demand. Requesters are free **by design**, because charging the demand side would suppress the liquidity that makes the platform valuable to supply. The pricing data already recorded (each offer's price/stance, the paid marker) is exactly what a future lead-fee or commission model would need — without operating payments now.

5. How the system is built

Three tiers with deliberately few moving parts:

```
Browser (Hebrew, RTL)
   │  reads = React Server Components   ·   writes = Server Actions
   ▼
Next.js 16 (App Router) on Vercel        ← no separate API server
   │  every call carries the signed-in user's JWT
   ▼
Supabase = PostgreSQL (RLS) + Auth + Storage
```

Reads are Server Components that query the database while rendering; writes are typed Server Actions. There is no custom backend and no REST API to secure.

6. The architecture — the database is the only authority

The thesis of the whole project: **every permission rule is enforced inside PostgreSQL** — Row Level Security policies, plus a small set of `SECURITY DEFINER` functions for the rules RLS can't express, plus guard triggers. The UI and Server Actions repeat those checks only for a friendly error; nothing depends on them for safety. A crafted request that bypasses the interface still hits the database with nothing but the caller's own identity, and is refused. That's why the app never uses the service-role key — even admin actions run as the signed-in user. The one design decision worth calling out: the `profiles` table is split into a public row and a **private** row (phone, coordinates, admin flag), because RLS is row-level, not column-level.

7. What the database looks like

Seven tables: `profiles` / `profiles_private`, `verification_applications`, `help_requests`, `offers`, `request_photos`, `ratings`. A request moves through a strict state machine — `open → has_offers → assigned → completed → rated` (plus a `cancelled` branch) — modelled as an enum with one timestamp column per state. Each RLS policy carries a comment naming the product rule it enforces (e.g. sealed bids: an offer is readable only by its owner and the request's owner).

8. The central processes

The core loop, all runnable on the live site (see the 5-minute review path below): post a request → helpers submit sealed offers with a pricing stance (fixed / volunteer / after-job) → the requester compares offers and picks one (an **atomic** assignment that closes all the others in a single transaction, and reveals the two phone numbers) → **both** sides confirm completion → the requester rates the helper, updating a public reputation page that never reveals who rated.

9. What tests were written

65 automated tests (64 Vitest + 1 Playwright end-to-end), all passing. The most important are the **integration tests that attack the database as the wrong user** — they attempt a forbidden action (making yourself admin, reading a competitor's sealed offer, editing someone else's row) and assert it is

denied. A happy-path-only test proves nothing; asserting denial is the point. The Playwright test drives the whole core loop through the real UI in two isolated browser sessions.

10. How scale was considered

Designed for tens-to-hundreds of users. The heaviest read is the feed: it fetches a capped 200 open requests, computes distance in TypeScript (Haversine — no PostGIS, no paid maps), sorts in memory, and paginates 12 per page. Every hot query has a matching index; list queries select explicit columns (no over-fetching); all reads are server-side with no client cache. Every limit has a named successor (e.g. DB-side distance with keyset pagination when the 200 cap bites). The honest observation: what breaks first at scale isn't the database — it's the **human admin verification queue**.

11. How security was considered

Four enforcement layers, and only the last is trusted: middleware, UI, Server Action guards, and the **database**. Authentication is Supabase Auth (email + password, bcrypt-hashed; sessions in HttpOnly/Secure/SameSite cookies, the JWT on every call). Authorization is RLS + the SECURITY DEFINER functions. The public anon key is safe to ship **because** RLS is the authority; the service-role key is in zero lines of app code. XSS, CSRF, and SQL injection are structurally neutralized (React auto-escaping, same-origin encrypted Server Actions, parameterized queries only). The security document keeps an honest list of remaining risks (manual identity review, request coordinates readable by signed-in users) with a named improvement for each.

12. What I'd improve with more time

In-app chat between the matched pair (they exchange phone numbers today); SMS OTP to actually verify the phone; in-app notifications so users don't have to refresh; and requester-side reputation to close the trust symmetry fully. All were deliberate cuts to keep the MVP small, clean, and secure.

Quick access (demo accounts)

All demo accounts on the live site use the password `12345678`:

Email	Role	Useful for
`dana@ihelp.demo`	Requester (verified)	Post a request, compare offers, rate
`yossi@ihelp.demo`	Professional helper (verified + badge)	Submit an offer
`admin@ihelp.demo`	Admin	Approve verifications, hide requests
`noa@ihelp.demo`	Not verified	See the verification gate that blocks actions

> **Tip:** to play requester and helper at once, use **two separate browsers** (or a normal + a private window) — not two tabs, which share the login cookie.

> **Is the site awake?** Open https://ihelp-roan.vercel.app/api/health — a `{"ok":true,"db":"reachable"}` response means the free-tier database is live. If not, wake it from the Supabase dashboard and wait 1–2 minutes.

5-minute review path (the core loop)

1. **Log in as the requester** — `dana@ihelp.demo` / `12345678`.
2. **Post a request** — "New help request" (`/requests/new`): title, description, category, confirm location, post. *A photo is optional.* Status becomes "Open".
3. **Log in as the helper** (second browser) — `yossi@ihelp.demo` / `12345678`, open the same request, submit an offer (pick a pricing stance). The helper does **not** see competing offers.
4. **Select** — back in Dana's browser, refresh (status "Has offers"), open the offer-comparison workbench, pick the offer. The assignment is atomic and closes the others; the phone numbers are now revealed to the two parties only.
5. **Both confirm** — each side clicks "Confirm the help was completed"; only when both have does the request become "Completed".
6. **Rate** — Dana rates the helper; his public reputation page updates without revealing who rated.

Bonus — the verification gate: log in as `noa@ihelp.demo` (unverified) and try to post — you're redirected to the verification page, proving permissions are enforced, not just hidden.

Where each submission item is

Item	Where
Product specification	`product-spec.md`
Technical design	`technical-design.md`
Testing specification	`testing-spec.md`
Test code	`test-code/` (also `tests/` , `e2e/` , `lib/*.test.ts` in the repo)
Basic scale	`scale.md`
Basic security	`security.md`
Running locally	`README.md` ("Running locally")
Live site	https://ihelp-roan.vercel.app
GitHub repository	https://github.com/YuvGev00/ihelp

Deeper reference (in the repository's `docs/` folder): architecture, an internal guide with the decision index, a file-by-file reference, and a map of each course concept to how it was implemented and where.